SPAD EN 1 SLIDE · STRUCTURED PROMPT-DRIVEN ENGINEERING

Construir software con IA, con control

Pensar, validar, ejecutar: un método para que el código que escribe la IA se pueda revisar, explicar y auditar.

El problema →
La IA escribe código muy rápido. Sin método, nadie decide el diseño, faltan pruebas y no quedan evidencias. La velocidad se vuelve deuda y riesgo.

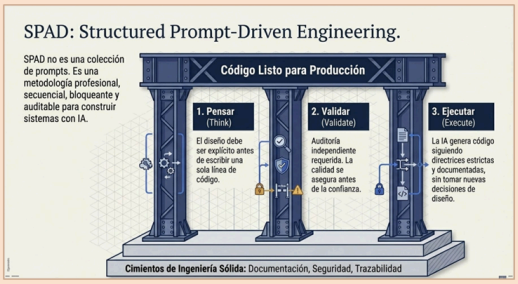
11 fases que bloquean →
Del contexto (fase 0) al plan, pruebas, código, revisiones y versión (fase 10). Sin validar, no se avanza. Trabajo pequeño: versión reducida.

Cada IA, un solo papel →
Una IA planifica, otra construye y otra distinta revisa. La que construye no decide el diseño. Ninguna IA se aprueba a sí misma.

GO · GO con cambios · NO-GO →
Cada revisión da un veredicto. Lo acepta una persona, que además revisa el código. Un NO-GO bien fundado es el método funcionando.

Todo queda escrito →
Contextos con las reglas de la compañía y del proyecto, un tema por trabajo y registro de qué modelo hizo cada artefacto y quién lo validó.

Encaje con SEVEN-G y SPHERES →
SPHERES: dónde y con qué ambición. SEVEN-G: qué se construye, con qué riesgo y cómo se mide. SPAD: cómo se construye (fases 4 y 5).



SPAD no promete cifras ni certifica: deja evidencias para que su compañía mida y audite lo que la IA construye.

→ Qué es SPAD | → Guía operativa: fases y roles | → SEVEN-G 53 · Construcción con IA | → Qué es SPHERES

SPAD · Versión 0.x, en revisión · CC BY 4.0 · Fernando García Varela seachad.github.io/AI_CONSULTING/ 1 / 1

Notas

SPAD (Structured Prompt-Driven Engineering, ingeniería estructurada guiada por instrucciones) es la metodología de Fernando García Varela para construir software con ayuda de la inteligencia artificial de forma gobernada. No es una colección de instrucciones para pedir código: es una forma de trabajar secuencial, bloqueante y auditable. La ilustración lo resume en tres pilares: pensar (el diseño es explícito antes de escribir una línea de código), validar (una revisión independiente) y ejecutar (la IA escribe el código sin tomar decisiones de diseño).

El problema. La IA genera código plausible a gran velocidad. Sin estructura, esa velocidad produce decisiones que nadie ha tomado de forma consciente, pruebas escasas, vulnerabilidades que nadie ve y deuda técnica. Ejemplo: un equipo pide código hasta que la demostración funciona; en producción falla y nadie sabe por qué el sistema está construido así. Por qué importa: el problema no es la IA, es la falta de método, y la IA lo amplifica.

Las fases. El ciclo principal tiene once fases, de la 0 a la 10: contexto y objetivo; plan, sin código; revisión del plan; guía de código; estrategia de pruebas; implementación; implementación de pruebas; revisión de pruebas; revisión del código; correcciones mínimas; y gestión de versiones, que es la entrega a operación. Ninguna fase empieza hasta que la anterior ha dejado su artefacto y una persona lo ha validado. Por qué importa: un error de arquitectura detectado en la revisión del plan cuesta una conversación; detectado en producción cuesta un incidente.

Los roles. Una persona orquesta: fija el objetivo, valida cada salida y tiene la última palabra. La IA planificadora diseña y no escribe código. La IA constructora escribe el código y no decide el diseño. La IA revisora, distinta de la que construye, evalúa y no corrige. La IA correctora aplica correcciones mínimas. Es la separación de funciones de cualquier proceso serio: quien hace no se aprueba a sí mismo. La IA revisora no es el Auditor de IA de SEVEN-G, que es una persona.

Los veredictos. Cada revisión termina con GO (se avanza), GO con cambios (se ajusta y se revisa de nuevo) o NO-GO (se vuelve atrás). Un GO con problemas graves abiertos es una autoaprobación y se descarta.

Además, una persona competente revisa el código antes de incorporarlo; en código sensible (pagos, datos personales, límites de actuación de agentes), una segunda persona. La política de validación añade una regla: una respuesta que se salta su fase se descarta entera, aunque el código funcione. Un NO-GO válido vale más que un GO inválido.

Contextos y registros. Antes de trabajar se cargan dos contextos: el global, con las reglas de la compañía (por ejemplo, ningún dato personal real se envía a un proveedor externo de IA), y el de proyecto. Cada trabajo tiene un tema, por ejemplo pdf_export, que agrupa todos sus artefactos. Cada artefacto registra qué modelo lo produjo y qué persona lo validó. Por qué importa: meses después, cualquiera puede reconstruir por qué algo se construyó así, y una auditoría encuentra las evidencias en un solo lugar.

Versión reducida. Para cambios acotados, sin datos personales ni dinero, la versión reducida (SPAD Lite) agrupa las fases en tres artefactos: plan revisado, entrega revisada y versión. No se recorta la separación entre la IA que construye y la que revisa, ni la validación humana. Si el software lleva IA dentro, SPAD añade una evaluación del comportamiento del sistema.

Encaje con los otros marcos. SPHERES ayuda a decidir dónde aporta la IA (nueve esferas) y con qué ambición (Optimizar, Aumentar, Transformar). SEVEN-G decide qué iniciativa se construye, con qué riesgo y cómo se mide su valor. SPAD gobierna cómo se construye el software, sobre todo en las fases 4 (Diseño de la solución) y 5 (Entrega y validación) de SEVEN-G (documento 53). Los veredictos de SPAD no sustituyen a las puertas de decisión de SEVEN-G: sus artefactos son insumos de las evidencias. SEVEN-G puede usarse con SPAD o con otro método documentado equivalente.

Lo que SPAD no hace: no promete cifras de mejora (cada organización mide las suyas con las métricas del proceso del documento 08) y no certifica; la autoevaluación de conformidad la hace la propia organización.

Pregunta para el aula: si su equipo usa IA para programar, ¿quién decide hoy el diseño y quién revisa lo que la IA escribe?

FUENTES Y ENLACES

- **SPAD 00 · Qué es SPAD y para qué sirve** https://seachad.github.io/AI_CONSULTING/SPAD/html/es/00_SPAD_Que_es_y_para_que_sirve.html
- **SPAD 01 · Guía operativa (fases, roles, veredictos, versión reducida)** https://seachad.github.io/AI_CONSULTING/SPAD/html/es/01_SPAD_Guia_operativa.html
- **SPAD 02 · Contextos, temas y registro de artefactos** https://seachad.github.io/AI_CONSULTING/SPAD/html/es/02_SPAD_Contextos_temas_y_registro.html
- **SPAD 03 · Política de validación** https://seachad.github.io/AI_CONSULTING/SPAD/html/es/03_SPAD_Politica_de_validacion.html
- **SEVEN-G 53 · Construcción de soluciones con IA** https://seachad.github.io/AI_CONSULTING/SEVEN-G/html/es/53_SEVEN-G_Construccion_de_soluciones_con_IA.html#relacion-entre-seven-g-y-spad
- **SPHERES 00 · Qué es SPHERES y para qué sirve** https://seachad.github.io/AI_CONSULTING/SPHERES/html/es/00_SPHERES_Que_es_y_para_que_sirve.html
- **Qué es SPAD** https://seachad.github.io/AI_CONSULTING/SPAD/html/es/00_SPAD_Que_es_y_para_que_sirve.html
- **Guía operativa: fases y roles** https://seachad.github.io/AI_CONSULTING/SPAD/html/es/01_SPAD_Guia_operativa.html
- **SEVEN-G 53 · Construcción con IA** https://seachad.github.io/AI_CONSULTING/SEVEN-G/html/es/53_SEVEN-G_Construccion_de_soluciones_con_IA.html
- **Qué es SPHERES** https://seachad.github.io/AI_CONSULTING/SPHERES/html/es/00_SPHERES_Que_es_y_para_que_sirve.html

AVISO LEGAL

Aviso legal: ayuda metodológica genérica y gratuita, «tal cual» y con fines informativos; no es asesoramiento jurídico ni está revisada jurídicamente para ningún caso. Cada organización la adapta y es responsable de su revisión legal y de su cumplimiento regulatorio; puede no recoger cambios normativos recientes. En la máxima medida permitida por la ley, el autor no asume responsabilidad alguna por los efectos de su aplicación. No otorga certificación de ningún tipo. Detalle en el documento 93.

CC BY 4.0 (contenidos) · MIT (código) · Fernando García Varela · seachad.github.io/AI_CONSULTING/